ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN CÔNG NGHỆ TRI THỨC

BÁO CÁO ĐỒ ÁN 01

Đề tài: HỆ MÃ HÓA RSA

Môn học: Nhập môn mã hóa – mật mã - 23CNTThuc2

Sinh viên thực hiện:

Lưu Vĩnh Phát - 23127448

Giáo viên hướng dẫn:

Thầy Nguyễn Đình Thúc

Thầy Nguyễn Văn Quang Huy

Thầy Trịnh Văn Minh

Ngày 5 tháng 11 năm 2025



Mục lục

1 Kiểm tra số nguyên tố **1**

1.1 Biểu diễn và lưu trữ số nguyên lớn 1

1.2 Các phép toán phụ trợ trên số lớn 2

1.3 Các phép toán modulo 5

1.4 Các phép toán modulo 6

1.5 Thuật toán Miller–Rabin kiểm tra số nguyên tố 7

1.6 Tổng kết phần Prime 8

2 Sinh khóa RSA **9**

2.1 Quy trình tổng quát sinh khóa 9

2.2 Tính toán $\varphi(N)$ và kiểm tra tính nghịch đảo 10

2.3 Tính nghịch đảo modular bằng Euclid mở rộng 11

2.4 Xuất khóa kết quả 12

2.5 Tổng kết phần Sinh khóa 13

3 Mã hóa RSA - Mã hóa và Giải mã **13**

3.1 Giới thiệu chung 13

3.2 Các hàm modulo và cơ chế hoạt động 14

3.3 Luồng xử lý trong chương trình 17

3.4 Tổng kết phần $y = x^k \bmod N$ 17

Danh sách hình vẽ

1 Biểu diễn số nguyên lớn bằng các limb 64-bit 1
2 Quá trình cộng và carry trong số lớn 2
3 Quá trình trừ với mượn bit 3
4 Quá trình dịch trái (nhân 16) 3
5 Quá trình dịch phải (chia 2) trong biểu diễn số lớn 4
6 Quá trình cộng và nhân modulo 6
7 Sơ đồ tổng quát quy trình sinh khóa RSA 10
8 Sơ đồ thuật toán Euclid mở rộng trong quá trình tìm d 12

1 Kiểm tra số nguyên tố

1.1 Biểu diễn và lưu trữ số nguyên lớn

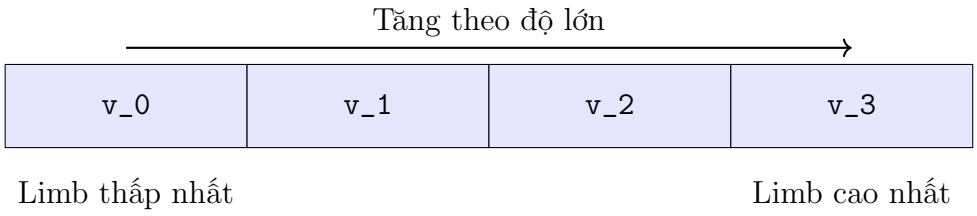
Để xử lý các số nguyên lớn (ví dụ, 512 bit trở lên), chương trình sử dụng cách chia nhỏ số thành các đoạn 64 bit gọi là **limb**, và lưu từng limb vào một mảng theo thứ tự **little-endian**.
Mỗi số được biểu diễn dưới dạng:

$$x = v_0 \cdot 2^{64 \cdot 0} + v_1 \cdot 2^{64 \cdot 1} + v_2 \cdot 2^{64 \cdot 2} + \dots$$

Ví dụ: Số $x = 2^{64} + 5$ được lưu thành mảng:

$$[v_0, v_1] = [5, 1]$$

Việc sử dụng cấu trúc này giúp dễ dàng thao tác dịch bit, cộng trừ và thực hiện các phép toán nhị phân.



Mỗi phần tử `v_i` là một **limb 64-bit**, được lưu theo thứ tự **little-endian**: limb thấp nhất ở chỉ số nhỏ nhất. Số nguyên lớn được biểu diễn như:

$$x = v_0 \cdot 2^{64 \cdot 0} + v_1 \cdot 2^{64 \cdot 1} + v_2 \cdot 2^{64 \cdot 2} + \dots$$

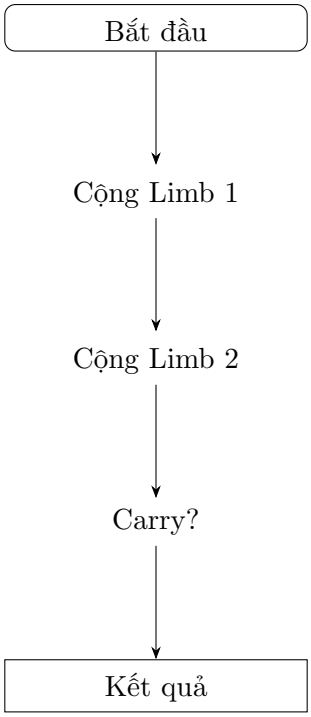
Hình 1: Biểu diễn số nguyên lớn bằng các limb 64-bit

—

1.2 Các phép toán phụ trợ trên số lớn

Cộng và cộng nhỏ

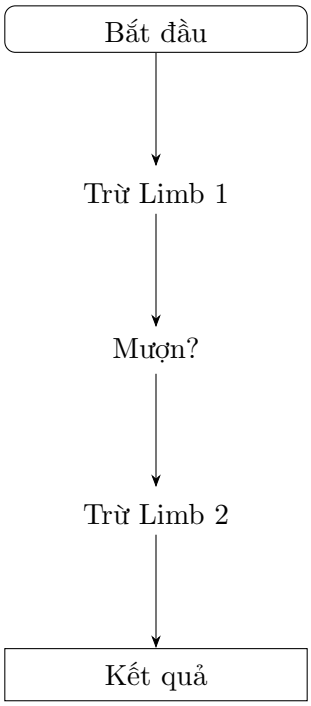
Phép cộng được thực hiện từng limb một, từ thấp lên cao, với cơ chế lan truyền số nhớ (carry) khi xảy ra tràn bit. Trường hợp cộng thêm một số nhỏ (0–15) sử dụng thuật toán tương tự nhưng tối ưu hơn cho hiệu suất.



Hình 2: Quá trình cộng và carry trong số lớn

Trừ

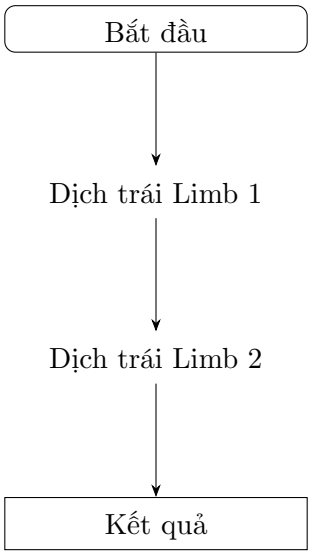
Phép trừ hai số lớn xử lý tình huống mượn bằng cách bổ sung 2^{64} vào limb hiện tại nếu giá trị tại đó nhỏ hơn số bị trừ. Quá trình mượn tiếp tục lan sang các limb cao hơn nếu cần.



Hình 3: Quá trình trừ với mượn bit

Dịch trái (nhân 16)

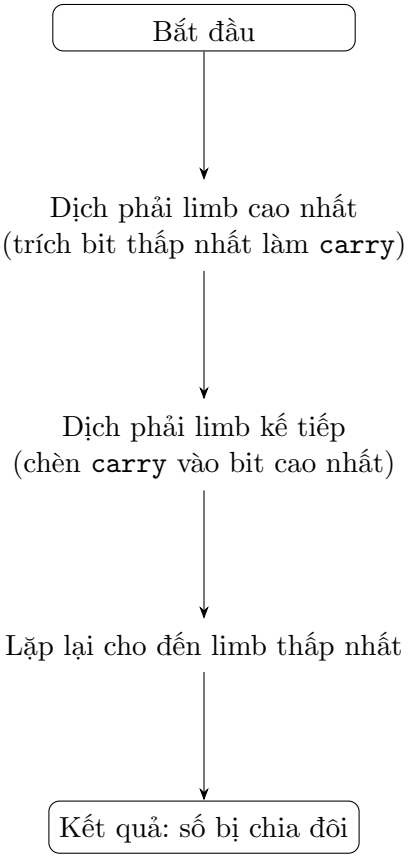
Dịch trái toàn bộ các limb 4 bit tương đương với nhân số lớn với 16. Bit tràn từ các limb thấp được đẩy lên các limb cao hơn để đảm bảo tính chính xác.



Hình 4: Quá trình dịch trái (nhân 16)

Dịch phải (chia 2)

Dịch phải từng limb 1 bit, sử dụng bit thừa từ limb cao hơn làm phần dư cho limb kế tiếp. Kỹ thuật này tương đương phép chia nguyên cho 2 trên số lớn. Mỗi lần dịch, bit thấp nhất của limb cao hơn (`carry`) sẽ được nối vào bit cao nhất của limb thấp hơn, đảm bảo toàn bộ số lớn được chia đúng theo cơ chế nhị phân.



Hình 5: Quá trình dịch phải (chia 2) trong biểu diễn số lớn

Ví dụ minh họa: Giả sử số lớn được lưu trong hai limb 64-bit theo thứ tự **little-endian**:

$$v_0 = \texttt{0xFFFFFFFFFFFFFFFF}, \quad v_1 = \texttt{0x0000000000000001}$$

Biểu diễn này tương ứng với:

$$x = v_0 + v_1 \cdot 2^{64} = (2^{64} - 1) + 2^{64} = 2^{65} - 1$$

Thực hiện lệnh `shr1()` (dịch phải 1 bit), thuật toán tiến hành:

1. Duyệt từ limb cao (v_1) xuống limb thấp (v_0).

2. Với $v_1 = 1$:

$$v_1' = v_1 \gg 1 = 0, \quad \texttt{carry} = 1$$

3. Với $v_0 = 0xFFFFFFFFFFFFFFFF$:

$$v_0' = (v_0 \gg 1) \mid (\texttt{carry} \ll 63) = 0x7FFFFFFFFFFFFFFF \mid 0x8000000000000000 = 0xFFFFFFF$$

Kết quả cuối cùng:

$$v_0' = \texttt{0xFFFFFFFFFFFFFFFF}, \quad v_1' = \texttt{0x0000000000000000}$$

tức là:

$$x' = v_0' + v_1' \cdot 2^{64} = 2^{64} - 1 = \left\lfloor \frac{2^{65} - 1}{2} \right\rfloor$$

Hàm `shr1()` vì vậy cho kết quả chính xác tương đương phép chia nguyên cho 2 trên toàn bộ số lớn, đồng thời bảo toàn tính đúng đắn giữa các limb thông qua cơ chế truyền bit `carry`.

1.3 Các phép toán modulo

Các phép toán **modulo** trên số lớn là cần thiết để thực hiện thuật toán kiểm tra số nguyên tố Miller–Rabin.

Cộng modulo và nhân modulo

- Phép cộng modulo đảm bảo kết quả nằm trong khoảng $[0, \text{mod})$ bằng cách:

$$\text{if } (a + b \geq \text{mod}) \Rightarrow \text{return } a + b - \text{mod}$$

- Nhân modulo được thực hiện bằng phương pháp **double-and-add**, tận dụng việc:

$$(a \cdot b) \bmod m = \sum_{i=0}^{n} b_i \cdot (a \cdot 2^i) \bmod m$$

Trong đó b_i là bit thứ i của số b.

—

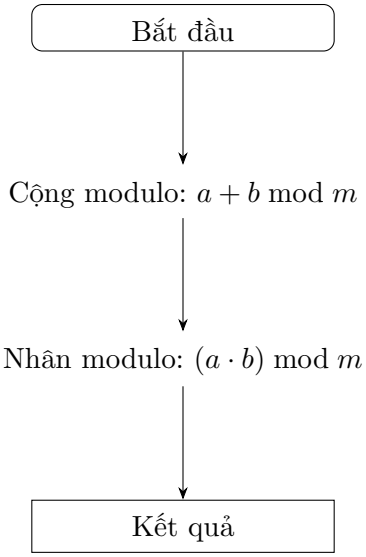
1.4 Các phép toán modulo

Cộng modulo và nhân modulo

Cộng modulo kiểm tra tổng của hai số lớn vượt quá modulus và trừ modulus nếu cần, nhằm đảm bảo kết quả nằm trong đoạn $[0, \text{mod})$. Phép nhân modulo được triển khai theo phương pháp nhân nhị phân (double-and-add), phù hợp với đại diện nhị phân của số nhân để tăng hiệu quả và kiểm soát tràn.

Lũy thừa modulo

Lũy thừa modulo sử dụng kỹ thuật bình phương và nhân lặp (square-and-multiply), xử lý mỗi bit của số mũ từ thấp đến cao. Với mỗi bit, nếu bit đang xét là 1, tích hiện tại được nhân thêm cơ số. Quá trình nhân và bình phương đều áp dụng modulo tại mỗi bước để giữ giá trị trong giới hạn.



Hình 6: Quá trình cộng và nhân modulo

Tất cả các phép toán đều có độ phức tạp tuyến tính theo số lượng limb, tức $O(n)$ với n là số limb (mỗi limb 64-bit).

1.5 Thuật toán Miller–Rabin kiểm tra số nguyên tố

Thuật toán Miller–Rabin là phương pháp xác suất được sử dụng rộng rãi để kiểm tra tính nguyên tố của các số lớn, đặc biệt phù hợp với môi trường tính toán không dùng thư viện ngoài như trong đồ án này.

Ý tưởng chính

Dựa trên định lý: nếu n là số nguyên tố lẻ và a là một số nguyên sao cho $1 < a < n-1$, thì:

$$a^{n-1} \equiv 1 \mod n$$

và tồn tại $r \in \mathbb{N}$ sao cho:

$$a^{d \cdot 2^r} \equiv -1 \mod n$$

với $n - 1 = 2^s \cdot d$, d lẻ.
Nếu không tìm được a nào thỏa mãn hai điều kiện trên, n chắc chắn không phải số nguyên tố.

Quy trình triển khai

1. Nếu n chẵn hoặc nhỏ hơn 3, xử lý trực tiếp.
2. Viết $n - 1$ dưới dạng $2^s \cdot d$, với d lẻ.
3. Duyệt qua một tập cơ sở kiểm tra gồm các số nguyên nhỏ, ví dụ:

```
{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}
```

4. Với mỗi cơ sở a:
   - Tính $x = a^d \mod n$
   - Nếu $x = 1$ hoặc $x = n - 1$, n có thể là số nguyên tố với cơ sở đó.
   - Nếu không, bình phương x tối đa $s - 1$ lần:
     - Nếu xuất hiện $x = n - 1$, tiếp tục với cơ sở kế tiếp.
     - Nếu gặp $x = 1$ trước khi có $n - 1$, n là hợp số (witness).

– Nếu không có $x = n - 1$ sau $s - 1$ lần, n là hợp số.

Cài đặt tối ưu

Thuật toán được tối ưu để sử dụng hoàn toàn số lớn do người dùng cài đặt thủ công:

- Sử dụng hàm `powMod` để tính $a^d \bmod n$ với hiệu suất cao.
- Tránh nhân số lớn trực tiếp: dùng phương pháp nhân nhị phân (double-and-add).
- Giảm số lần kiểm tra bằng cách chỉ chọn các cơ sở phù hợp nhỏ hơn n.
- Nếu n vượt quá a, bỏ qua cơ sở đó vì không còn ý nghĩa kiểm tra.

Độ chính xác và hiệu quả

Dù Miller–Rabin là thuật toán xác suất, việc chọn đủ cơ sở (12 số nhỏ đầu tiên) đảm bảo độ chính xác cao đối với số có kích thước đến 512 bit.
Các phép toán chính được cài đặt hoàn toàn trên cấu trúc số lớn tùy chỉnh giúp đảm bảo tính tự chủ, không phụ thuộc thư viện thứ ba, phù hợp yêu cầu đồ án.

1.6 Tổng kết phần Prime

Phần này trình bày quy trình cài đặt kiểm tra số nguyên tố cho số lớn:

- Tự xây dựng cấu trúc dữ liệu số lớn dạng limb 64-bit.
- Cài đặt các phép toán cơ bản trên số lớn.
- Cài đặt phép cộng, trừ, chia 2, nhân 16, modulo và lũy thừa modulo không dùng thư viện ngoài.
- Triển khai thuật toán Miller–Rabin với nhiều cơ sở kiểm tra để đảm bảo độ chính xác cao.

Bằng việc kết hợp các thành phần trên, chương trình kiểm tra số nguyên tố nhanh chóng và chính xác cho đầu vào lên tới 512-bit.

2 Sinh khóa RSA

Mục tiêu. Phần này mô tả quá trình sinh khóa riêng d trong hệ mật RSA dựa trên hai số nguyên tố p, q và số mũ công khai e. Chương trình thực hiện toàn bộ pipeline: đọc dữ liệu đầu vào, tính $\varphi(N)$, kiểm tra tính nghịch đảo modular của e, và cuối cùng sinh ra khóa d bằng thuật toán Euclid mở rộng.

—

2.1 Quy trình tổng quát sinh khóa

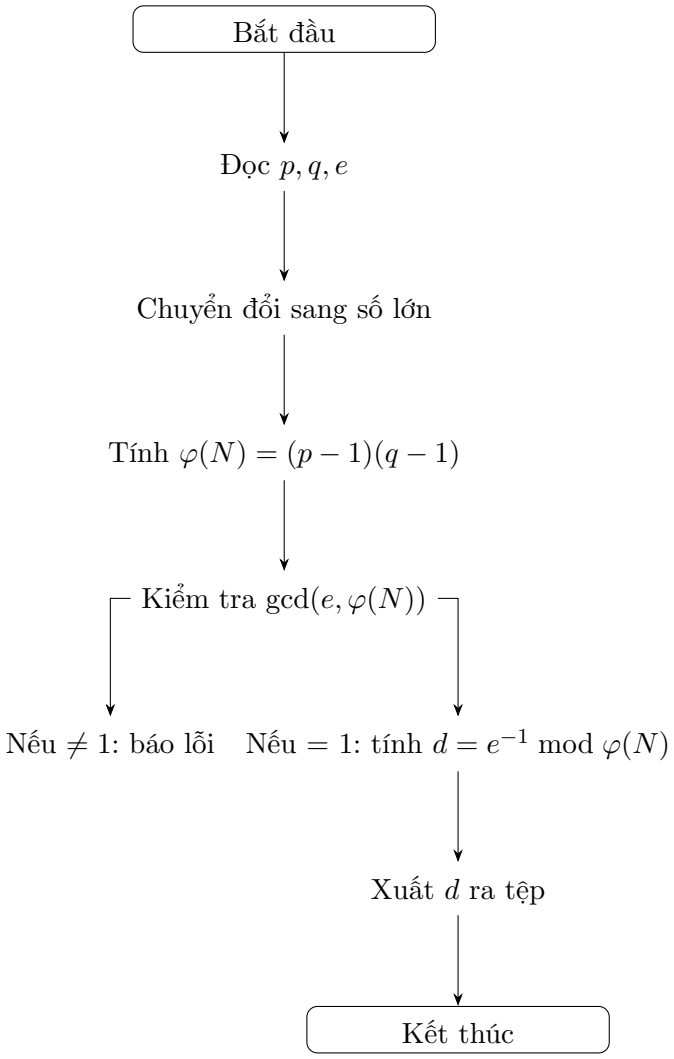
Các bước chính

Thuật toán sinh khóa RSA tuân theo trình tự sau:

1. **Đọc dữ liệu đầu vào:** nhập các giá trị p, q, e dưới dạng chuỗi *hex little-endian*.
2. **Chuyển đổi sang số lớn:** mỗi chuỗi được xử lý bởi hàm `parse_hex_le`, biến đổi thành số nguyên lớn (BigInt).
3. **Tính các giá trị trung gian:**

$$N = p \times q, \quad \varphi(N) = (p-1)(q-1)$$

4. **Kiểm tra điều kiện nghịch đảo:** tính $g = \gcd(e, \varphi(N))$. Nếu $g \neq 1$, dừng và thông báo lỗi (không tồn tại d).
5. **Tính khóa riêng:** áp dụng Euclid mở rộng để tìm $d = e^{-1} \bmod \varphi(N)$.
6. **Xuất kết quả:** ghi d ra tệp ở dạng *hex little-endian*.



Hình 7: Sơ đồ tổng quát quy trình sinh khóa RSA

—

2.2 Tính toán $\varphi(N)$ và kiểm tra tính nghịch đảo

Sau khi đã xác định được hai số nguyên tố p và q, bước tiếp theo là tính $\varphi(N)$:

$$\varphi(N) = (p-1) \times (q-1)$$

Phép nhân được thực hiện bằng hàm `mul()` hoạt động trên các limb 64-bit, đảm bảo độ chính xác tuyệt đối mà không cần thư viện hỗ trợ ngoài.

Sau đó, chương trình tính:

$$g = \gcd(e, \varphi(N))$$

Sử dụng thuật toán **Stein (Binary GCD)**, chương trình loại bỏ các thừa số 2 chung, rồi lặp phép trừ và dịch phải đến khi một trong hai giá trị trở về 0. Nếu $g \neq 1$, chương trình ghi `-1` ra file kết quả — nghĩa là không thể sinh khóa riêng d.

—

2.3 Tính nghịch đảo modular bằng Euclid mở rộng

Nguyên lý toán học

Nghịch đảo modular của e theo mô-đun $\varphi(N)$ là số d thỏa:

$$e \times d \equiv 1 \pmod{\varphi(N)}$$

Tức là tồn tại các số nguyên x, y sao cho:

$$e \times x + \varphi(N) \times y = 1$$

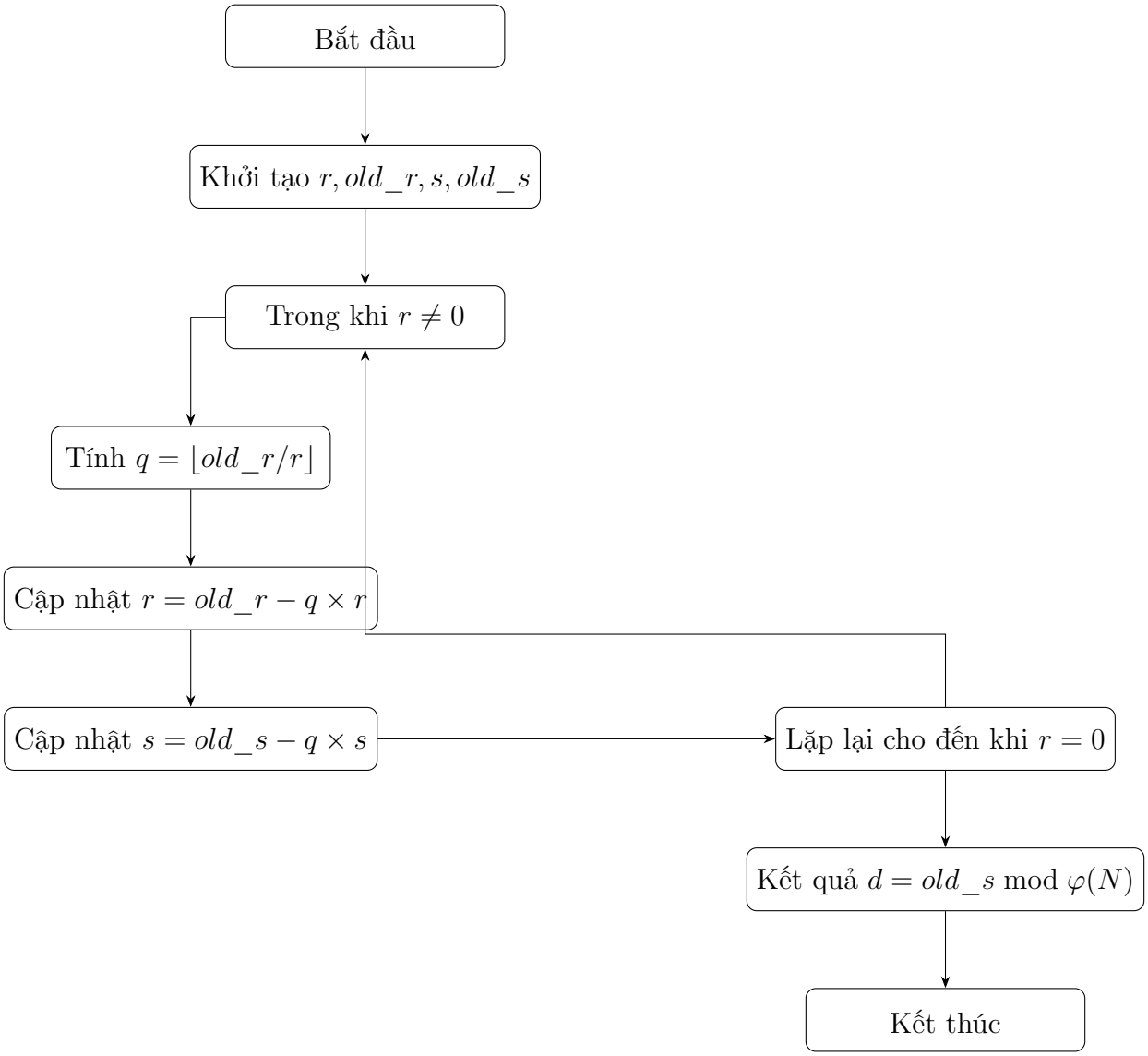
Khi đó, x chính là giá trị d cần tìm.
Để tính x, thuật toán **Euclid mở rộng** được sử dụng với việc duy trì hai cặp giá trị phần dư (r, old_r) và hệ số tương ứng (s, old_s) qua từng bước chia.

Quy trình hoạt động

1. Khởi tạo:
$$old_r = \varphi(N), \quad r = e, \quad old_s = 0, \quad s = 1$$
2. Trong khi $r \neq 0$:
   - Tính thương $q = \lfloor old_r / r \rfloor$
   - Cập nhật:
$$(old_r, r) \leftarrow (r, old_r - q \times r)$$
$$(old_s, s) \leftarrow (s, old_s - q \times s)$$

3. Khi vòng lặp kết thúc, old_s là nghịch đảo modular mong muốn. Nếu $old_s < 0$, cộng thêm $\varphi(N)$ để đưa về miền dương.

—



Hình 8: Sơ đồ thuật toán Euclid mở rộng trong quá trình tìm d

—

2.4 Xuất khóa kết quả

Sau khi tính được d, chương trình:

- Chuyển d sang dạng *hex big-endian*.

- Đảo chuỗi để thu được định dạng *little-endian*.
- Loại bỏ các ký tự '0' dư ở cuối để thu gọn kết quả.
- Ghi chuỗi cuối cùng vào tệp đầu ra `.out`.

Ví dụ: $d = 0x00000000000001BB \Rightarrow$ hex little-endian: `BB10000000000000`

—

2.5 Tổng kết phần Sinh khóa

Phần sinh khóa RSA đảm nhiệm vai trò trung tâm trong hệ thống, gắn kết toàn bộ các thành phần:

- Sử dụng kết quả của phần kiểm tra số nguyên tố để đảm bảo p, q hợp lệ.
- Tính $\varphi(N)$ và kiểm tra điều kiện nghịch đảo giữa e và $\varphi(N)$.
- Cài đặt chính xác thuật toán Euclid mở rộng để tìm khóa riêng d mà không cần thư viện hỗ trợ.
- Đảm bảo định dạng xuất đúng chuẩn little-endian theo yêu cầu.

Nhờ cấu trúc tính toán tách biệt, phần này có thể được tái sử dụng cho mọi cặp (p, q, e) hợp lệ, tạo thành nền tảng hoàn chỉnh cho việc sinh và quản lý khóa trong hệ mật RSA.

3 Mã hóa RSA - Mã hóa và Giải mã

3.1 Giới thiệu chung

Trong phần này, chương trình thực hiện bài toán tổng quát:

$$y = x^k \bmod N,$$

với ba số nguyên dương (N, k, x) được cho trước. Kết quả y là **số nguyên dương nhỏ nhất** thoả mãn quan hệ đồng dư:

$$y \equiv x^k \pmod{N}.$$

Như vậy, về cơ bản, cơ chế này bao quát cả hai quy trình mã hóa và giải mã trong RSA, vì chúng đều quy về cùng công thức trên. Nhiệm vụ của chương trình là thực hiện phép tính này với số nguyên có thể dài đến hàng trăm bit mà không sử dụng thư viện ngoài, chỉ dựa vào các hàm tự xây dựng trong cấu trúc `Big`.

—

3.2 Các hàm modulo và cơ chế hoạt động

Phép toán modulo được chia thành ba cấp độ chính: **cộng modulo**, **nhân modulo**, và **lũy thừa modulo**. Tất cả đều hoạt động trên biểu diễn số lớn dạng limb 64-bit đã trình bày trong các phần trước.

1. Cộng modulo – `addMod(a, b, mod)`

Hàm này tính tổng:

$$(a + b) \bmod m$$

theo các bước:

1. Cộng hai số lớn a và b bằng `add()`.
2. Nếu kết quả vượt quá m, thực hiện phép trừ `sub()` để đưa về miền $[0, m)$.

Cách làm này giúp tránh tràn bit khi cộng và bảo đảm kết quả luôn nhỏ hơn modulus.

Ví dụ:

$$a = 9, \quad b = 8, \quad m = 10$$

$$a + b = 17 \Rightarrow (a + b) \bmod m = 17 - 10 = 7.$$

Trong chương trình, các giá trị này được lưu và tính toán ở cấp độ 64-bit limb, đảm bảo mở rộng cho các số có hàng nghìn bit.

—

2. Nhân modulo – `mulMod(a, b, mod)`

Phép nhân modulo được triển khai theo phương pháp **nhân nhị phân (double-and-add)**, tức là không thực hiện phép nhân toàn phần, mà thay bằng chuỗi các phép cộng và nhân đôi có kiểm soát modulo.

Thuật toán hoạt động như sau:

1. Khởi tạo kết quả $res = 0$.
2. Trong khi $b \neq 0$:
   - Nếu bit thấp nhất của b bằng 1, cập nhật $res = (res + a) \bmod m$.
   - Nhân đôi a (dịch trái 1 bit) rồi lấy modulo: $a = (2a) \bmod m$.
   - Dịch phải b một bit (tức là chia b cho 2).

Diễn tiến từng bước:

$$\text{Khởi tạo: } res = 0,\ a = 7,\ b = 13,\ m = 10.$$

Vòng	b (thập phân)	LSB(b)	Cập nhật $res \leftarrow (res + a) \bmod m$	Cập nhật $a \leftarrow (2a) \bmod m$	$b \leftarrow b \gg 1$
1	13	1	$res = (0 + 7) \bmod 10 = 7$	$a = (2 \cdot 7) \bmod 10 = 4$	$b = 6$
2	6	0	res giữ $= 7$	$a = (2 \cdot 4) \bmod 10 = 8$	$b = 3$
3	3	1	$res = (7 + 8) \bmod 10 = 5$	$a = (2 \cdot 8) \bmod 10 = 6$	$b = 1$
4	1	1	$res = (5 + 6) \bmod 10 = 1$	$a = (2 \cdot 6) \bmod 10 = 2$	$b = 0$

Kết quả cuối: $res = 1 \ \Rightarrow\ 7 \cdot 13 \bmod 10 = 1$.

Thuật toán này đảm bảo không bao giờ phải lưu trữ giá trị trung gian lớn hơn m, nhờ đó giảm đáng kể bộ nhớ và tránh tràn.

Ví dụ:

$$a = 7, \quad b = 13, \quad m = 10$$

Ta có biểu diễn nhị phân của $b = (1101)_2$. Khi đó:

$$7 \times 13 \bmod 10 = (7 + 14 + 56) \bmod 10 = 91 \bmod 10 = 1.$$

Quy trình này trùng khớp với kết quả hàm `mulMod()`.

—

3. Lũy thừa modulo – `powMod(base, exp, mod)`

Đây là hàm trung tâm của chương trình, tính:

$$\texttt{powMod}(x, k, N) = x^k \bmod N$$

theo kỹ thuật **bình phương và nhân lặp (square-and-multiply)**.

1. Khởi tạo $res = 1$.
2. Trong khi $exp \neq 0$:
   - Nếu bit thấp nhất của exp là 1, thực hiện $res = (res \times base) \bmod mod$.
   - Cập nhật $base = (base \times base) \bmod mod$.
   - Dịch phải exp một bit.

Diễn tiến từng bước:

Khởi tạo: $res = 1,\ base = 4,\ k = 13,\ N = 497.$

Vòng	k (thập phân)	LSB(k)	$res \leftarrow (res \cdot base) \bmod N$	$base \leftarrow (base^2) \bmod N$
1	13	1	$res = (1 \cdot 4) \bmod 497 = 4$ $k \to 6$	$base = (4^2) \bmod 497 = 16$
2	6	0	res giữ $= 4$ $k \to 3$	$base = (16^2) \bmod 497 = 256$
3	3	1	$res = (4 \cdot 256) \bmod 497 = 30$ $k \to 1$	$base = (256^2) \bmod 497 = 429$
4	1	1	$res = (30 \cdot 429) \bmod 497 = 445$ $k \to 0$	$base = (429^2) \bmod 497 = ...$

Kết quả cuối: $res = 445 \Rightarrow 4^{13} \bmod 497 = 445$.

Cách này tận dụng tính chất nhị phân của số mũ, giúp giảm số phép nhân xuống mức $O(\log k)$ thay vì $O(k)$.

Ví dụ:

$$x = 4, \quad k = 13, \quad N = 497$$

Biểu diễn $k = (1101)_2$. Ta có:

$$4^1 \bmod 497 = 4$$

$$4^2 \bmod 497 = 16$$

$$4^4 \bmod 497 = 256$$

$$4^8 \bmod 497 = (256)^2 \bmod 497 = 429$$

$$4^{13} \bmod 497 = (4^8 \times 4^4 \times 4^1) \bmod 497 = (429 \times 256 \times 4) \bmod 497 = 445.$$

Kết quả: $y = 445$, đúng với đầu ra của chương trình.

—

3.3 Luồng xử lý trong chương trình

Từ ba giá trị đầu vào (N, k, x) ở dạng hex little-endian, chương trình thực hiện quy trình sau:

1. Đọc và chuyển đổi chuỗi hex sang số lớn bằng `parse_hex_le()`.
2. Thực hiện $y = \texttt{powMod}(x, k, N)$.
3. Chuyển kết quả y ngược lại sang dạng hex little-endian.
4. Ghi kết quả vào tệp đầu ra.

—

3.4 Tổng kết phần $y = x^k \bmod N$

Phần này trình bày toàn bộ cơ chế xử lý phép tính modulo bậc cao, nền tảng cho mọi thao tác trong RSA:

- Hàm `addMod()` đảm bảo cộng đúng giới hạn modulo.
- Hàm `mulMod()` sử dụng nhân nhị phân để tránh tràn và tăng tốc độ.
- Hàm `powMod()` áp dụng thuật toán bình phương–nhân lặp, có độ phức tạp $O(\log k)$.
- Tất cả các phép tính đều hoạt động trên biểu diễn số lớn tự định nghĩa bằng limb 64-bit.

Nhờ thiết kế này, chương trình có thể xử lý các số N có độ dài đến 1024 bit trong thời gian giới hạn, đảm bảo kết quả chính xác và ổn định.

Tài liệu

[1] "Multiply large integers under large modulo." *GeeksforGeeks*. https://www.geeksforgeeks.org/dsa/multiply-large-integers-under-large-modulo/

[2] "Extended Euclidean Algorithm." *CP-Algorithms*. https://cp-algorithms.com/algebra/extended-euclid-algorithm.html

[3] "Miller–Rabin primality test." *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Miller%E2%80%93Rabin_primality_test

[4] "The Miller-Rabin Test." Stanford University – Number Theory Lecture Notes. https://crypto.stanford.edu/pbc/notes/numbertheory/millerrabin.html

[5] "Multiply large integers under large modulo." *GeeksforGeeks*. https://www.geeksforgeeks.org/dsa/multiply-large-integers-under-large-modulo/ :contentReference[oaicite:0]index=0

[6] "Binary Exponentiation – Algorithms for Competitive Programming." *CP-Algorithms*. https://cp-algorithms.com/algebra/binary-exp.html :contentReference[oaicite:1]index=1

[7] "Modular exponentiation." *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Modular_exponentiation :contentReference[oaicite:2]index=2